

Bài tập 1: Tìm hiểu và trình bày về các lớp bài toán

1. Lớp bài toán P (Polynomial time)

- **Định nghĩa:** Là tập hợp các bài toán quyết định (decision problems) có thể giải được bằng máy Turing đơn định (deterministic Turing machine) trong thời gian đa thức $O(n^k)$.
- **Ý nghĩa:** Đây là những bài toán có thể giải quyết một cách hiệu quả trên máy tính thực tế (ví dụ: thuật tìm kiếm nhị phân, sắp xếp, tìm đường đi ngắn nhất Dijkstra).

2. Lớp bài toán NP (Nondeterministic Polynomial time)

- **Định nghĩa:** Là tập hợp các bài toán quyết định có thể giải được bằng máy Turing không đơn định trong thời gian đa thức. Nói cách khác, nếu một bài toán thuộc NP, ta có thể *kiểm tra* tính đúng đắn của một lời giải (chứng chỉ - certificate) trong thời gian đa thức bằng máy đơn định.
- **Ý nghĩa:** Tìm lời giải thì khó, nhưng kiểm tra lời giải đúng hay sai thì nhanh.

3. Lớp bài toán Co-NP (Complement of NP)

- **Định nghĩa:** Là lớp các bài toán bổ pho của NP. Một bài toán quyết định $A \in \text{co-NP}$ khi và chỉ khi bài toán phủ định của nó (bài toán có câu trả lời ngược lại: "Không" thành "Có" và ngược lại) thuộc NP.
- **Ví dụ:** Bài toán kiểm tra một số có phải là số nguyên tố hay không thuộc Co-NP (vì bài toán tìm xem nó có phải hợp số không thuộc NP).

4. Mối liên hệ giữa P, NP và Co-NP

- Hệ quả hiển nhiên: $P \subseteq NP$ và $P \subseteq \text{co-NP}$. Do đó, $P \subseteq (NP \cap \text{co-NP})$.
- Người ta giả định rằng $P = NP \cap \text{co-NP}$, nhưng chưa có chứng minh chính thức.

5. Lớp bài toán NP-hard (NP-khó)

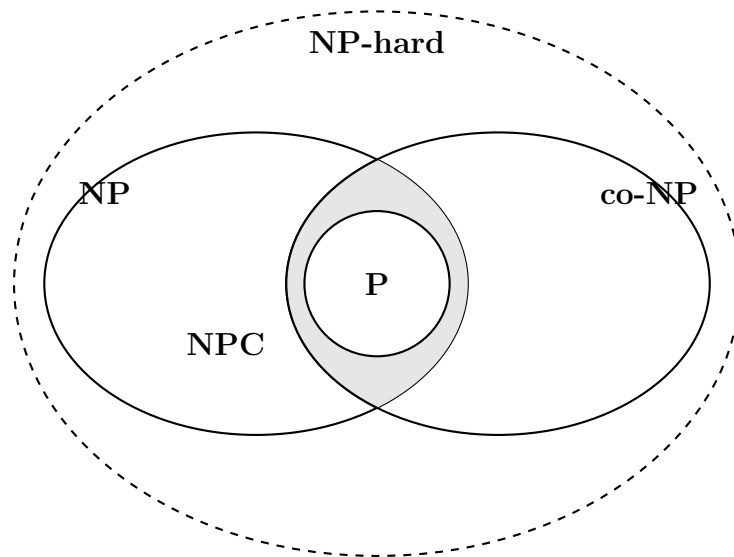
- **Định nghĩa:** Một bài toán H (không nhất thiết phải là bài toán quyết định) được gọi là NP-hard nếu mọi bài toán trong NP đều có thể quy dẫn đa thức (polynomial-time reducible) về H .
- **Ý nghĩa:** Các bài toán NP-hard "khó ít nhất bằng" các bài toán khó nhất trong NP.

6. Lớp bài toán NPC (NP-complete - NP đầy đủ)

- **Định nghĩa:** Một bài toán C được gọi là NP-complete nếu nó thỏa mãn hai điều kiện:
 - (a) $C \in NP$
 - (b) $C \in \text{NP-hard}$
- **Ý nghĩa:** Đây là những bài toán khó nhất trong tập NP. Nếu giải được một bài toán NPC trong thời gian đa thức, ta sẽ giải được toàn bộ các bài toán khác trong NP với thời gian đa thức.

7. Mối liên hệ giữa P, NP, Co-NP, NP-Hard và NPC

- Nếu giả định rằng $P \neq NP$ (giả định phổ biến nhất hiện nay), mối quan hệ cấu trúc giữa các lớp bài toán này được biểu diễn trực quan qua sơ đồ Venn dưới đây:



8. Bài toán $P = NP$?

- Đây là một trong những câu hỏi lớn nhất chưa có lời giải của khoa học máy tính hiện đại (một trong 7 bài toán Thiên niên kỷ).
- **Nếu $P = NP$:** Mọi bài toán có lời giải dễ dàng kiểm tra được thì cũng sẽ giải được một cách nhanh chóng. Cấu trúc mật mã hiện đại (như RSA) sẽ sụp đổ.
- **Nếu $P \neq NP$:** Thế giới toán học và máy tính giữ nguyên trật tự hiện tại; việc tìm kiếm lời giải cho các bài toán tối ưu lớn (như NPC) về cơ bản là không thể thực hiện chính xác trong thời gian ngắn, buộc phải dùng thuật toán xấp xỉ (approximation) hoặc Heuristic.

Bài tập 2: Bản chất của thuật toán và độ phức tạp bài toán

1. Định nghĩa thuật toán nhanh, hiệu quả và độ phức tạp

- **Thuật toán nhanh (Fast Algorithm):** Là thuật toán có thời gian thực thi ngắn (số lượng phép toán cần thực hiện ít) trên thực tế hoặc có hàm số biểu diễn độ phức tạp thời gian tăng trưởng chậm khi kích thước đầu vào n tiến ra vô cùng. Thông thường, các thuật toán chạy trong thời gian đa thức $O(n^k)$ với k nhỏ (ví dụ: $O(n)$, $O(n \log n)$) được coi là nhanh.
- **Thuật toán hiệu quả (Efficient Algorithm):** Không chỉ quan tâm đến tốc độ, thuật toán hiệu quả là sự cân bằng tối ưu giữa *Độ phức tạp thời gian (Time Complexity)* và *Độ phức tạp không gian (Space Complexity - bộ nhớ chiếm dụng)*, đồng thời tận dụng tốt cấu trúc dữ liệu để giải quyết bài toán trong giới hạn tài nguyên cho phép.
- **Độ phức tạp của thuật toán (Algorithm Complexity):** Là hàm số đo lường lượng tài nguyên (thời gian hoặc bộ nhớ) mà thuật toán tiêu tốn dưới dạng một hàm số của kích thước đầu vào n . Thường được biểu diễn qua ký pháp Big-O (O), Big-Omega (Ω), và Big-Theta (Θ).

2. Khái niệm bài toán "Dễ" và "Khó"

- **Bài toán "Dễ" (Tractable Problems):** Là các bài toán tồn tại thuật toán giải chính xác trong thời gian đa thức $O(n^k)$ ở trường hợp xấu nhất. Chúng thuộc lớp bài toán P .
- **Bài toán "Khó" (Intractable Problems):** Là các bài toán mà cho đến nay chưa ai tìm được thuật toán thời gian đa thức để giải chính xác trong trường hợp tổng quát. Thời gian chạy của các thuật toán chính xác cho bài toán này thường là hàm mũ $O(2^n)$ hoặc giai thừa $O(n!)$. Chúng thường thuộc lớp NP -hard hoặc NPC .

3. Ví dụ, thiết kế, phân tích và đánh giá thuật toán

A. Ví dụ về Bài toán "Dễ": Tìm cặp số có tổng cho trước (Two Sum Variant)

Phát biểu bài toán: Cho một dãy số nguyên A gồm n phần tử đã được sắp xếp tăng dần và một số nguyên S . Hãy kiểm tra xem có tồn tại hai chỉ số $i \neq j$ sao cho $A[i] + A[j] = S$ hay không.

Thiết kế thuật toán: Sử dụng kỹ thuật *Hai con trỏ* (*Two Pointers*).

- Đặt con trỏ $left = 1$ và $right = n$.
- Vòng lặp dừng khi $left \geq right$. Trong mỗi bước, tính $sum = A[left] + A[right]$.
- Nếu $sum = S$: Trả về True.
- Nếu $sum < S$: Tăng $left$ lên 1 (để tăng tổng).
- Nếu $sum > S$: Giảm $right$ đi 1 (để giảm tổng).

Phân tích độ phức tạp:

- **Thời gian:** Trong mỗi bước của vòng lặp, ít nhất một con trỏ dịch chuyển một đơn vị. Số bước tối đa là n . Mỗi bước chỉ tốn thời gian hằng số $O(1)$. Do đó, độ phức tạp thời gian là $O(n)$.
- **Không gian:** Chỉ sử dụng các biến con trỏ phụ, độ phức tạp không gian là $O(1)$.

Đánh giá: Thuật toán này cực kỳ *hiệu quả* và *nhANH*. So với cách tiếp cận duyệt trau (Brute Force) tốn $O(n^2)$, việc tận dụng tính chất mảng đã sắp xếp giúp hạ độ phức tạp xuống tuyến tính $O(n)$, minh chứng cho một bài toán "Dễ".

B. Ví dụ về Bài toán "Khó": Bài toán người giao hàng thu nhỏ (Travelling Salesperson Problem - TSP)

Phát biểu bài toán: Cho một đồ thị đầy đủ gồm n thành phố và ma trận chi phí/khoảng cách giữa các thành phố. Một người giao hàng xuất phát từ thành phố 1, phải đi qua tất cả các thành phố khác đúng một lần rồi quay về thành phố xuất phát. Tìm hành trình có tổng chi phí nhỏ nhất.

Thiết kế thuật toán (Nhánh cận - Branch and Bound): Để giải chính xác bài toán này, ta xây dựng cây quyết định duyệt qua tất cả các hoán vị của các thành phố (Duyệt toàn bộ tốn $O(n!)$). Để tối ưu, ta áp dụng *Nhánh cận*: Tại mỗi bước duyệt, tính toán một giá trị cận dưới (Lower Bound) của chi phí hành trình hiện tại. Nếu cận dưới này lớn hơn hoặc bằng chi phí của cấu hình tốt nhất đã tìm thấy trước đó (*Best Cost*), ta lập tức tĩa nhánh (không duyệt sâu xuống nhánh đó nữa).

Phân tích độ phức tạp:

- **Thời gian:** Trong trường hợp xấu nhất (các hàm chặn không hiệu quả), thuật toán vẫn phải duyệt qua hầu hết các trạng thái, độ phức tạp thời gian tiệm cận $O(n \cdot 2^n)$ (nếu dùng quy hoạch động) hoặc $O(n!)$ (nếu dùng quay lui thuần túy).
- **Không gian:** Tốn $O(n^2)$ để lưu ma trận khoảng cách và $O(n)$ hoặc $O(2^n)$ để lưu trạng thái tùy cách cài đặt.

Đánh giá: TSP là một bài toán *NP-hard* điển hình. Khi n tăng lên khoảng $n = 50$ hoặc $n = 100$, việc tìm lời giải chính xác bằng Nhánh cận hay Quy hoạch động trên máy tính siêu mạnh cũng mất hàng năm trời (bùng nổ tổ hợp). Đây là một bài toán "Khó". Trên thực tế, người ta buộc phải chấp nhận từ bỏ tính chính xác hoàn toàn để dùng các thuật toán *Heuristic* (như Thuật toán tham lam, Di truyền - Genetic Algorithm, hay Tối ưu hóa bầy đàn - PSO) để tìm lời giải xấp xỉ gần đúng trong thời gian đa thức nhằm đạt tính *hiệu quả*.

Bài tập 3: Khả năng giải quyết các bài toán trong thời gian đa thức

1. Trả lời khẳng định

Không thể giải quyết tất cả các bài toán trong thời gian đa thức. Khoa học máy tính hiện đại đã chứng minh toán học rằng tồn tại những giới hạn tuyệt đối mà máy tính không thể vượt qua, bất kể phần cứng có phát triển mạnh mẽ đến đâu hay thuật toán có tối ưu như thế nào.

2. Các nguyên nhân cốt lõi

Có hai lý do chính giải thích vì sao thời gian đa thức không thể bao phủ mọi bài toán:

A. Sự tồn tại của các bài toán có độ phức tạp hàm mũ cố định (Provably Intractable Problems)

Có những bài toán mà người ta đã chứng minh được rằng thuật toán tốt nhất có thể giải chúng **bắt buộc** phải tốn thời gian lũy thừa hoặc giai thừa, không đơn thuần là do chúng ta chưa tìm ra thuật toán nhanh hơn, mà do kích thước đầu ra hoặc số lượng trạng thái phải duyệt mang tính bùng nổ tổ hợp.

- **Định lý tăng trưởng cấp số mũ (Exponential Hierarchy):** Theo Định lý phân cấp thời gian (Time Hierarchy Theorem), nếu cho máy tính nhiều thời gian hơn, nó sẽ giải được nhiều bài toán hơn. Cụ thể, lớp bài toán P (thời gian đa thức) là một tập con thực sự của lớp $EXPTIME$ (thời gian hàm mũ $O(2^{n^k})$).

- **Ví dụ điển hình:** Bài toán tìm nước đi tối ưu trong cờ vua hoặc cờ vây trên bàn cờ tổng quát kích thước $n \times n$. Số lượng trạng thái của bàn cờ tăng theo hàm mũ so với kích thước n , khiến việc tìm kiếm lời giải chính xác bắt buộc phải tốn chi phí thời gian siêu đa thức ($EXPTIME$ -complete).

B. Sự tồn tại của các bài toán không thể giải được (Undecidable Problems)

Vượt lên trên cả vấn đề thời gian đa thức hay hàm mũ, có những bài toán mà **không tồn tại bất kỳ một thuật toán nào** có thể giải được chúng một cách chính xác trong thời gian hữu hạn. Nếu một bài toán không thể giải được, thì việc bàn luận về thời gian đa thức là vô nghĩa.

- **Bài toán dừng (The Halting Problem):** Được Alan Turing chứng minh vào năm 1936. Bài toán phát biểu: "*Thiết kế một thuật toán tổng quát để kiểm tra xem một chương trình máy tính bất kỳ có bị lặp vô hạn hay sẽ dừng lại với một đầu vào cho trước*". Turing đã dùng phương pháp phản chứng (phương pháp chéo Cantor) để khẳng định rằng một thuật toán tổng quát như vậy không bao giờ tồn tại.

C. Thách thức lớn từ giả thuyết $P \neq NP$

Ngay cả khi chỉ xét trong tập hợp các bài toán quyết định thực tế thuộc lớp NP (những bài toán kiểm tra lời giải thì nhanh nhưng tìm lời giải thì chưa biết có nhanh không), hầu hết các nhà khoa học máy tính đều tin vào giả thuyết:

$$P \neq NP \quad (1)$$

Nếu giả thuyết này đúng (điều mà thế giới đang thừa nhận dù chưa có chứng minh chính thức), toàn bộ các bài toán thuộc lớp NPC (NP -complete) và NP -hard như tối ưu hóa mạng lưới, lập lịch sản xuất, mã hóa dòng dữ liệu... đều **không thể** giải được trong thời gian đa thức.

3. Kết luận

Khả năng giải quyết bài toán bằng máy tính bị chặn bởi hai rào cản lớn:

1. **Rào cản về tính toán được (Computability):** Giới hạn những gì thuật toán có thể và không thể làm (ví dụ: Halting Problem).
2. **Rào cản về tính khả thi (Tractability):** Giới hạn giữa những bài toán giải được một cách thực tế (P) và những bài toán bùng nổ tổ hợp ($EXPTIME$).

Do đó, việc tìm kiếm một "thuật toán vạn năng" chạy trong thời gian đa thức cho mọi bài toán là điều bất khả thi về mặt toán học.

Bài tập 4: Tính phổ quát của lớp bài toán NP-Hard

1. Trả lời khẳng định

Không phải bất kỳ bài toán nào cũng là NP -hard. Khẳng định cho rằng mọi bài toán đều là NP -hard là hoàn toàn sai lầm cả về mặt trực giác thực tế lẫn định nghĩa toán học chặt chẽ. Lớp NP -hard chỉ chứa các bài toán có độ khó tối thiểu bằng các bài toán khó nhất trong tập hợp NP .

2. Các luận điểm chứng minh

A. Dựa trên định nghĩa về phép quy dẫn đa thức (Polynomial-time Reduction)

Theo định nghĩa, một bài toán H được gọi là NP -hard nếu **mọi** bài toán $A \in NP$ đều có thể quy dẫn thời gian đa thức về H (ký hiệu: $A \leq_P H$).

- Quy dẫn $A \leq_P H$ có nghĩa là nếu ta có một "hộp đen" giải được bài toán H trong thời gian đa thức, ta có thể dùng nó như một thuật toán con để giải quyết bài toán A trong thời gian đa thức.
- Nếu H là một bài toán cực kỳ đơn giản (ví dụ: bài toán kiểm tra một số nguyên có phải là số chẵn hay không), ta không thể dùng nó để giải quyết các bài toán phức tạp trong NP như bài toán Thỏa mãn công thức logic (SAT) hay Tìm chu trình Hamilton. Do đó, bài toán kiểm tra số chẵn không thể là NP -hard.

B. Sự tồn tại của các bài toán "Dễ" (Lớp P)

Nếu giả thuyết $P \neq NP$ đúng (giả thuyết được hầu hết các nhà khoa học thừa nhận), các bài toán thuộc lớp P (giải được trong thời gian đa thức) **không thể** là NP -hard.

- Các bài toán cơ bản như: Tìm kiếm nhị phân, Sắp xếp mảng (*Merge Sort*, *Quick Sort*), Tìm đường đi ngắn nhất (*Dijkstra*) đều nằm trong lớp P .
- Nếu một bài toán thuộc lớp P mà lại là NP -hard, điều đó đồng nghĩa với việc ta có thể dùng thuật toán thời gian đa thức của bài toán đó để giải **tất cả** các bài toán trong NP trong thời gian đa thức. Hệ quả là $P = NP$. Do đó, dưới điều kiện $P \neq NP$, không có bài toán nào trong P có thể là NP -hard.

C. Biểu diễn qua sơ đồ cấu trúc độ phức tạp

Xét trong không gian các bài toán quyết định (dưới giả định $P \neq NP$), ta có sự phân tách ranh giới rất rõ ràng về độ khó giữa các lớp bài toán:

$$P \subset NP \subset \text{Toàn bộ không gian bài toán}$$

Trong đó, NP -hard nằm ở vùng "phía trên", đại diện cho ranh giới của những bài toán khó. Các bài toán thuộc lớp P nằm ở vùng lõi dưới cùng (vùng các bài toán dễ giải quyết) và hoàn toàn không giao với vùng NP -hard.

3. Kết luận

Lớp NP -hard không phải là một tập hợp chứa tất cả mọi bài toán, mà nó hoạt động như một **thước đo cận dưới về độ khó**. Tồn tại vô số các bài toán nằm ngoài lớp NP -hard vì độ phức tạp tính toán của chúng thấp hơn nhiều so với tiêu chuẩn "khó ít nhất bằng các bài toán khó nhất trong NP ".

Bài tập 5: Bản chất và hiểu lầm về lớp bài toán NP

1. Trả lời khẳng định

Khẳng định "Lớp bài toán NP là lớp bài toán không thể giải được trong thời gian đa thức" là ****hoàn toàn sai lầm****.

Lớp NP không phải là tập hợp các bài toán "không phải đa thức" (*Non-Polynomial*). Thực tế, lớp NP chứa rất nhiều bài toán giải được trong thời gian đa thức, và bản thân định nghĩa của nó cũng dựa trên nền tảng thời gian đa thức.

2. Các luận điểm chứng minh

A. Chữ NP không có nghĩa là "Không đa thức" (Non-Polynomial)

- NP là viết tắt của **Nondeterministic Polynomial time** (Thời gian đa thức không đơn định).
- Định nghĩa chuẩn tắc: Một bài toán quyết định thuộc lớp NP nếu nó có thể được giải bởi một **máy Turing không đơn định** (Nondeterministic Turing Machine) trong thời gian đa thức.
- Định nghĩa tương đương trên máy tính thực tế (máy đơn định): Một bài toán thuộc NP nếu khi ta đưa ra một ứng viên lời giải (gọi là chứng chỉ - *certificate*), máy tính có thể **kiểm tra (verify)** xem lời giải đó đúng hay sai trong thời gian đa thức.

B. Mỗi quan hệ bao hàm $P \subseteq NP$

- Một kết quả toán học đã được chứng minh và không phụ thuộc vào giả thuyết là:

$$P \subseteq NP \quad (2)$$

- Điều này có nghĩa là mọi bài toán thuộc lớp P (giải được trong thời gian đa thức trên máy tính thông thường) **đều mặc nhiên thuộc lớp NP **.
- **Lý do:** Nếu chúng ta có thể tự tìm ra lời giải cho một bài toán trong thời gian đa thức (thuộc P), thì việc kiểm tra một lời giải có sẵn lại càng dễ dàng thực hiện được trong thời gian đa thức (thuộc NP).
- **Ví dụ:** Bài toán sắp xếp mảng, tìm kiếm nhị phân, hay tìm đường đi ngắn nhất (*Dijkstra*) đều giải được trong thời gian đa thức, và chúng đều nằm trong lớp NP .

C. Bản chất vùng xám của NP và giả thuyết $P \neq NP$

- Tập hợp NP có thể chia làm hai phần chính dưới góc nhìn của giả thuyết $P \neq NP$:
 1. Phần đã biết cách giải nhanh: Chính là lớp P .
 2. Phần chưa biết có giải nhanh được không: Bao gồm lớp NPC (NP-complete) như bài toán SAT, Tô màu đồ thị, Chu trình Hamilton...
- Ngay cả đối với các bài toán NPC , giới khoa học máy tính hiện nay cũng **chưa thể chứng minh** rằng chúng không thể giải được trong thời gian đa thức, mà họ mới chỉ **giả định** như vậy ($P \neq NP$). Nếu một ngày có ai đó chứng minh được $P = NP$, thì toàn bộ các bài toán trong NP đều giải được trong thời gian đa thức.

3. Kết luận

Tóm lại, lớp NP không phải là lớp các bài toán không thể giải trong thời gian đa thức. Trái lại:

- Nó ****chắc chắn chứa**** toàn bộ lớp P (các bài toán giải được trong thời gian đa thức).
- Đối với phần còn lại của NP (như NPC), câu hỏi liệu chúng có thể giải được trong thời gian đa thức hay không vẫn là một bài toán mở lớn nhất kỷ nguyên máy tính (P vs NP).

Bài tập 6: Hệ quả của giả thuyết $P = NP$ đối với lớp NPC

1. Trả lời khẳng định

Khẳng định "Nếu $P = NP$ thì $NPC \subseteq P$ " là **ĐÚNG**.

Đây là một hệ quả toán học logic hiển nhiên dựa trên định nghĩa về các lớp độ phức tạp tính toán và lý thuyết tập hợp.

2. Chứng minh và giải thích vì sao

Mệnh đề trên được chứng minh một cách chặt chẽ thông qua hai luận điểm sau:

A. Dựa trên quan hệ tập hợp (Set Inclusion)

Theo định nghĩa chuẩn tắc của khoa học máy tính:

- Lớp bài toán NPC (NP-complete) là tập hợp các bài toán khó nhất **nằm bên trong** lớp NP . Để một bài toán C thuộc lớp NPC , nó bắt buộc phải thỏa mãn điều kiện tiên quyết là: $C \in NP$.
- Do đó, ta luôn có mối quan hệ tập con bất biến sau:

$$NPC \subseteq NP \quad (3)$$

- Theo lý thuyết tập hợp cơ bản, nếu ta có hai tập hợp A và B thỏa mãn $A \subseteq B$, và nếu giả định $B = C$, thì hiển nhiên $A \subseteq C$.
- Áp dụng vào trường hợp này: Vì $NPC \subseteq NP$, nếu giả thuyết $P = NP$ xảy ra, ta chỉ cần thay thế tập hợp NP bằng tập hợp P . Hệ quả logic trực tiếp thu được là:

$$NPC \subseteq P \quad (4)$$

B. Dựa trên tính chất quy dẫn của lớp NPC (Polynomial-time Reduction)

Một góc nhìn khác sâu sắc hơn đến từ chính định nghĩa của tính đầy đủ NP (NP -complete):

- Một bài toán C thuộc NPC có tính chất vạn năng: **Mọi** bài toán thuộc NP đều có thể quy dẫn thời gian đa thức về C (ký hiệu $A \leq_P C, \forall A \in NP$).
- Điều này nghĩa là: Nếu ta tìm được thuật toán thời gian đa thức để giải quyết **chỉ một** bài toán bất kỳ thuộc lớp NPC , thì thuật toán đó có thể dùng để giải quyết **tất cả** các bài toán khác trong NP trong thời gian đa thức. Khi đó, toàn bộ lớp NP sẽ bị "kéo" về lớp P , tức là $P = NP$.

- Ngược lại, nếu ta giả định ngay từ đầu rằng $P = NP$, điều đó có nghĩa là các bài toán khó nhất trong NP (chính là lớp NPC) giờ đây đã tìm được thuật toán giải chính xác trong thời gian đa thức. Vì vậy, mọi bài toán thuộc NPC lúc này đều thỏa mãn định nghĩa của lớp P . Do đó, $NPC \subseteq P$ (thực chất lúc này $NPC = P \setminus \{\emptyset, \Sigma^*\}$).

3. Hệ quả thế giới thực nếu điều này xảy ra

Nếu giả thuyết $P = NP$ thành hiện thực và $NPC \subseteq P$:

- Các bài toán tối ưu hóa tổ hợp siêu khó đang thách thức nhân loại như: Lập lịch chuỗi cung ứng toàn cầu, dự đoán cấu trúc protein, toán học mật mã nền tảng của ngân hàng (RSA, AES) đều sẽ có thuật toán giải cực nhanh trong thời gian đa thức.
- Khái niệm bài toán "Khó" dạng bùng nổ tổ hợp trong phân khúc NP sẽ hoàn toàn biến mất, mọi bài toán kiểm tra được lời giải thì đều sẽ giải được một cách dễ dàng.

Bài tập 7: Hệ quả ngược từ mối quan hệ giữa NPC và P

1. Trả lời khẳng định

Khẳng định "Nếu $NPC \subseteq P$ thì $P = NP$ " là **ĐÚNG**.

Đây là một trong những định lý nền tảng và quan trọng nhất của lý thuyết độ phức tạp tính toán, khẳng định vai trò "chìa khóa" của lớp bài toán NP -complete.

2. Chứng minh và giải thích vì sao

Mệnh đề này được chứng minh một cách chặt chẽ dựa trên tính chất quy dẫn đa thức (Polynomial-time Reduction) của các bài toán NPC :

A. Bản chất vận năng của lớp NPC

Theo định nghĩa chuẩn tắc của Stephen Cook và Leonid Levin, một bài toán C được công nhận là NP -complete (NPC) khi và chỉ khi nó thỏa mãn hai điều kiện:

1. $C \in NP$
2. $C \in NP$ -hard, nghĩa là **mọi** bài toán $A \in NP$ đều có thể quy dẫn thời gian đa thức về C (ký hiệu: $A \leq_P C$).

Tính chất quy dẫn $A \leq_P C$ có nghĩa là: Nếu tồn tại một thuật toán giải quyết bài toán C trong thời gian đa thức, thì ta có thể biến đổi bất kỳ bài toán A nào trong NP về dạng của C , rồi dùng thuật toán của C để giải A cũng trong thời gian đa thức.

B. Các bước lập luận logic

- Giả sử ta có giả thiết: $NPC \subseteq P$. Điều này có nghĩa là mọi bài toán thuộc NPC đều có thuật toán giải chính xác trong thời gian đa thức.
- Lấy một bài toán cụ thể $C \in NPC$. Từ giả thiết trên, ta suy ra $C \in P$.
- Bây giờ, xét một bài toán bất kỳ A nằm trong tập hợp NP ($A \in NP$).
- Vì C là bài toán NP -complete, theo định nghĩa cấu trúc, bài toán A chắc chắn quy dẫn được về C trong thời gian đa thức ($A \leq_P C$).

- Do $C \in P$ (giải được trong thời gian đa thức) và $A \leq_P C$, thuật toán tổng quát để giải A sẽ bằng: $[Thời\ gian\ quy\ dẫn\ sang\ C] + [Thời\ gian\ giải\ bài\ toán\ C]$. Vì cả hai quá trình này đều tốn thời gian đa thức, tổng thời gian để giải A cũng là thời gian đa thức.
- Từ đó suy ra $A \in P$. Vì điều này đúng với **mọi** bài toán $A \in NP$, ta thu được kết luận:

$$NP \subseteq P \quad (5)$$

- Mặt khác, ta đã biết một sự thật toán học luôn đúng là $P \subseteq NP$. Từ mối quan hệ bao hàm kép ($NP \subseteq P$ và $P \subseteq NP$), ta đi đến kết luận cuối cùng:

$$P = NP \quad (6)$$

3. Ý nghĩa triết học và khoa học của định lý

Định lý này mang lại một ý nghĩa thực tiễn vô cùng lớn cho giới khoa học:

- Thay vì phải đi tìm cách chứng minh $P = NP$ cho hàng ngàn bài toán khác nhau trong lớp NP , các nhà toán học và tin học chỉ cần tập trung vào đúng **một** bài toán duy nhất thuộc lớp NPC (ví dụ như bài toán SAT, bài toán Chu trình Hamilton, hay bài toán Tô màu đồ thị).
- Nếu họ tìm được thuật toán thời gian đa thức cho chỉ một bài toán NPC đó, họ sẽ ngay lập tức chứng minh được $P = NP$ cho toàn bộ thế giới tính toán. Ngược lại, nếu chứng minh được một bài toán NPC bất kỳ bắt buộc phải tốn thời gian hàm mũ, họ sẽ khép lại thiên niên kỷ với lời giải $P \neq NP$.

4. Lưu ý nhỏ về trường hợp suy biến (Technical Caveat)

Về mặt toán học cực kỳ nghiêm ngặt, phép quy dẫn đa thức không thể áp dụng cho hai ngôn ngữ suy biến là tập rỗng $\emptyset$ và tập toàn vũ trụ Σ^* (vì không có câu trả lời "Có" hoặc "Không" để chuyển đổi). Do đó, nếu định nghĩa lớp P và NP chứa cả hai tập hợp này, thì mối quan hệ chính xác khi $NPC \subseteq P$ là $NP = P$ hoàn toàn đúng, ngoại trừ các trường hợp suy biến không có giá trị thực tế đó. Xét trên khía cạnh thuật toán thực nghiệm, mệnh đề "Nếu $NPC \subseteq P$ thì $P = NP$ " luôn luôn đúng.

Bài tập 8: Tìm hiểu, thiết kế và phân tích Thuật toán xấp xỉ

1. Lựa chọn bài toán và Thuật toán xấp xỉ

Bài toán Phủ đỉnh tối thiểu (Minimum Vertex Cover Problem): Cho một đồ thị vô hướng $G = (V, E)$. Một tập phủ đỉnh $C \subseteq V$ là tập hợp các đỉnh sao cho với mọi cạnh $(u, v) \in E$, ít nhất u hoặc v phải thuộc C . Mục tiêu là tìm tập phủ đỉnh C có kích thước $|C|$ nhỏ nhất. Đây là một bài toán thuộc lớp NPC .

Thuật toán xấp xỉ CLR (Dựa trên tập cạnh độc lập): Ý tưởng là chọn một cạnh bất kỳ, thêm cả hai đầu mút của nó vào tập phủ, sau đó xóa tất cả các cạnh liên thuộc với hai đỉnh này. Lặp lại cho đến khi không còn cạnh nào trên đồ thị.

[H] Thuật toán xấp xỉ tỷ số 2 cho Vertex Cover Đồ thị $G = (V, E)$ Tập phủ đỉnh $C \leftarrow \emptyset$ $E' \leftarrow E$ *Sao chép tập cạnh $E' \neq \emptyset$ Chọn một cạnh bất kỳ $(u, v) \in E'$ $C \leftarrow C \cup \{u, v\}$ Xóa khỏi E' tất cả các cạnh có đầu mút liên thuộc với u hoặc v C

Chứng minh tỷ số xấp xỉ (Approximation Ratio = 2): Gọi A là tập hợp các cạnh được chọn trong vòng lặp **while**. Do cách thuật toán hoạt động, không có hai cạnh nào trong A chung đỉnh với nhau (tập cạnh độc lập - *matching*). Để phủ được $|A|$ cạnh độc lập này, một thuật toán tối ưu (OPT) bắt buộc phải chọn ít nhất 1 đỉnh từ mỗi cạnh, suy ra $|C_{OPT}| \geq |A|$. Trong khi đó, thuật toán xấp xỉ chọn cả 2 đỉnh của mỗi cạnh trong A , do đó $|C| = 2|A|$. Từ đây ta có:

$$|C| = 2|A| \leq 2|C_{OPT}| \implies \frac{|C|}{|C_{OPT}|} \leq 2 \quad (7)$$

Thuật toán luôn trả về một lời giải không bao giờ tệ hơn 2 lần lời giải tối ưu.

2. Minh họa áp dụng thuật toán

Xét đồ thị $G = (V, E)$ với:

- Tập đỉnh: $V = \{1, 2, 3, 4, 5, 6\}$
- Tập cạnh: $E = \{(1, 2), (2, 3), (3, 4), (4, 5), (4, 6), (5, 6)\}$

Từng bước thực thi của thuật toán:

- **Khởi tạo:** $C = \emptyset$, $E' = \{(1, 2), (2, 3), (3, 4), (4, 5), (4, 6), (5, 6)\}$.
- **Bước 1:** Chọn ngẫu nhiên cạnh $(1, 2) \in E'$.
 - Cập nhật: $C = \{1, 2\}$.
 - Xóa các cạnh chứa đỉnh 1 hoặc 2. Tập cạnh còn lại: $E' = \{(3, 4), (4, 5), (4, 6), (5, 6)\}$.
- **Bước 2:** Chọn ngẫu nhiên cạnh $(3, 4) \in E'$.
 - Cập nhật: $C = \{1, 2, 3, 4\}$.
 - Xóa các cạnh chứa đỉnh 3 hoặc 4. Tập cạnh còn lại: $E' = \{(5, 6)\}$.
- **Bước 3:** Chọn cạnh còn lại duy nhất $(5, 6) \in E'$.
 - Cập nhật: $C = \{1, 2, 3, 4, 5, 6\}$.
 - Xóa các cạnh chứa đỉnh 5 hoặc 6. Lúc này $E' = \emptyset$. Vòng lặp kết thúc.

Kết quả xấp xỉ: Thuật toán trả về tập phủ $C = \{1, 2, 3, 4, 5, 6\}$ với kích thước $|C| = 6$.

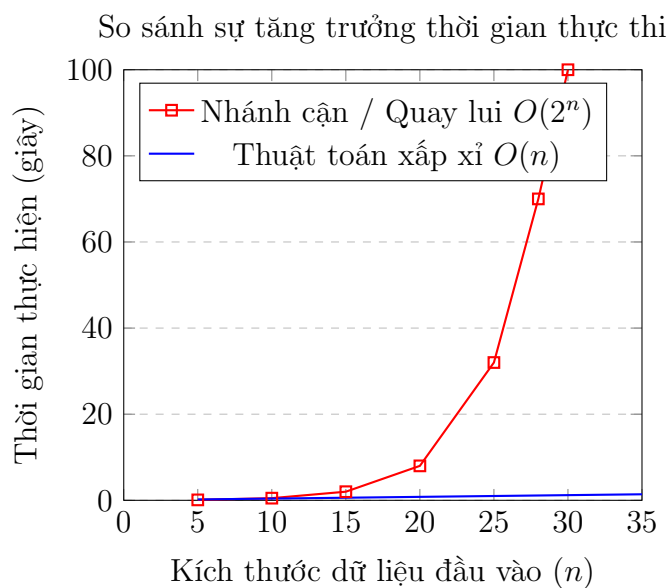
Đánh giá với lời giải tối ưu: Lời giải tối ưu thực tế cho đồ thị này là $C_{OPT} = \{2, 4, 5\}$ với kích thước $|C_{OPT}| = 3$ (đỉnh 2 phủ $(1, 2), (2, 3)$; đỉnh 4 phủ $(3, 4), (4, 5), (4, 6)$; đỉnh 5 phủ $(5, 6)$). Ta thấy tỷ số thực tế là $6/3 = 2$, hoàn toàn thỏa mãn chặn xấp xỉ.

3. So sánh với các kỹ thuật thiết kế thuật toán khác

Tiêu chí	Thuật toán Xấp xỉ (Approx)	Nhánh cận / Quay lui (Exact)
Tính chính xác	Trả về lời giải gần đúng, có cam kết sai số so với tối ưu (độ lỗi ≤ 2).	Luôn trả về lời giải chính xác tối ưu tuyệt đối.
Thời gian thực thi	Cực nhanh, chạy trong thời gian đa thức $O(V + E)$.	Rất chậm khi dữ liệu lớn, thời gian hàm mũ $O(2^V)$ (Bùng nổ tổ hợp).
Phạm vi áp dụng	Thích hợp với đồ thị kích thước lớn (hàng triệu đỉnh) trong thực tế.	Chỉ chạy được với đồ thị nhỏ ($V < 30$ đỉnh).

4. Đánh giá sự tăng trưởng thời gian thực hiện (Đồ thị)

Độ phức tạp thời gian của thuật toán xấp xỉ là tuyến tính $O(V + E)$, trong khi thuật toán chính xác (Nhánh cận) tốn thời gian hàm mũ $O(2^V)$. Biểu đồ dưới đây minh họa sự khác biệt rõ rệt về thời gian chạy khi kích thước dữ liệu n (số đỉnh V) tăng lên.



Nhận xét từ biểu đồ: Khi $n < 15$, thời gian chạy của cả hai phương pháp không có sự chênh lệch lớn. Tuy nhiên, khi $n \geq 25$, đường cong của thuật toán chính xác (màu đỏ) dựng đứng lên theo hàm mũ, gây ra hiện tượng nghẽn cổ chai hệ thống. Trong khi đó, thuật toán xấp xỉ (màu xanh) duy trì mức tăng trưởng tuyến tính cực kỳ ổn định, chứng minh tính hiệu quả vượt trội về mặt tài nguyên thời gian đối với các bài toán quy mô lớn.